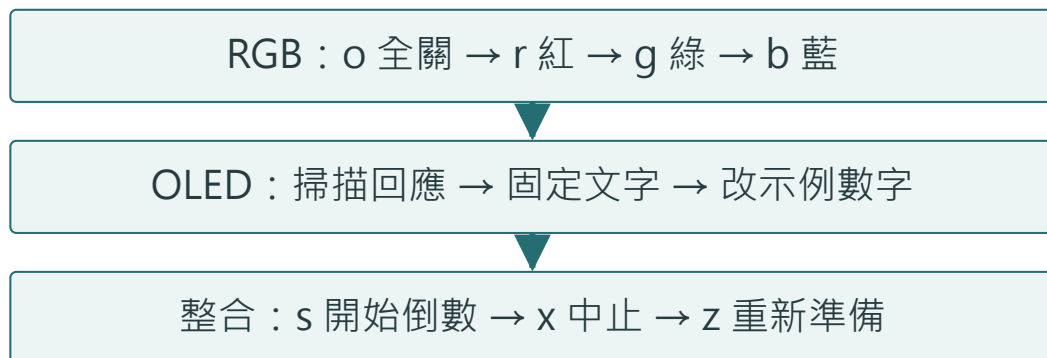


做一個看得到、停得下來的倒數器

先逐個模組做成功，再把它們接成同一個作品。

上週我們讓感測器回報資料。這週改練輸出：讓燈號與小螢幕告訴使用者，作品現在正在做什麼。



完成後，送出 s 會開始 30 秒倒數，紅綠燈每 3 秒輪換；送 x 可以中止。時間到，燈關閉，畫面顯示到期。

本週**不計遮光得分、不接舵機或電池**。螢幕上的 DEMO 數字由我們輸入，不是感測器計分。

我們照講義一起做，先看到結果，再解釋原理。整段程式放在後面，操作時先改設定，不必先讀完所有程式。

先拿 RGB 和 OLED

其他模組先移開，避免舊接線被新程式控制。

拿到桌上	用途
ESP32-S3、USB 資料線、電腦	上傳與文字命令
四針 RGB 模組	紅、綠、藍單色測試
四針 I2C OLED	顯示文字與倒數
麵包板、母對母及公對母線	連接訊號與電源
電表、原接線紀錄	課前核對與低壓排查
KY-018 與上週基準	最後才做光線干擾比較

1. 外接電源關閉、USB 拔除，確認 PWR 熄滅。
2. 拍照保存原作品，再拆下本週不用的線。
3. ESP32 留在桌面，麵包板 a~e 的第 3、6、15 列留空。
4. 今天不用電流檔或 10A 孔；通斷與 Ω 只在外部斷電時量。

課前需確認本批 RGB 的限流、各色電流與有效準位，以及 OLED 供電、SDA / SCL 電位及適用驅動。資料尚未確認，就先做程式與紙上判讀，不試接未知腳位。

先把自己這組的腳位寫清楚

GPIO 編號看板上標示，不是從第一根排針開始數。

下表是教師 BOARD-T01 曾有功能反應的接法，**不是整批器材的安全認證**。只有課前核對後確認適用，才依它填設定；不同板子要使用自己的已確認表。

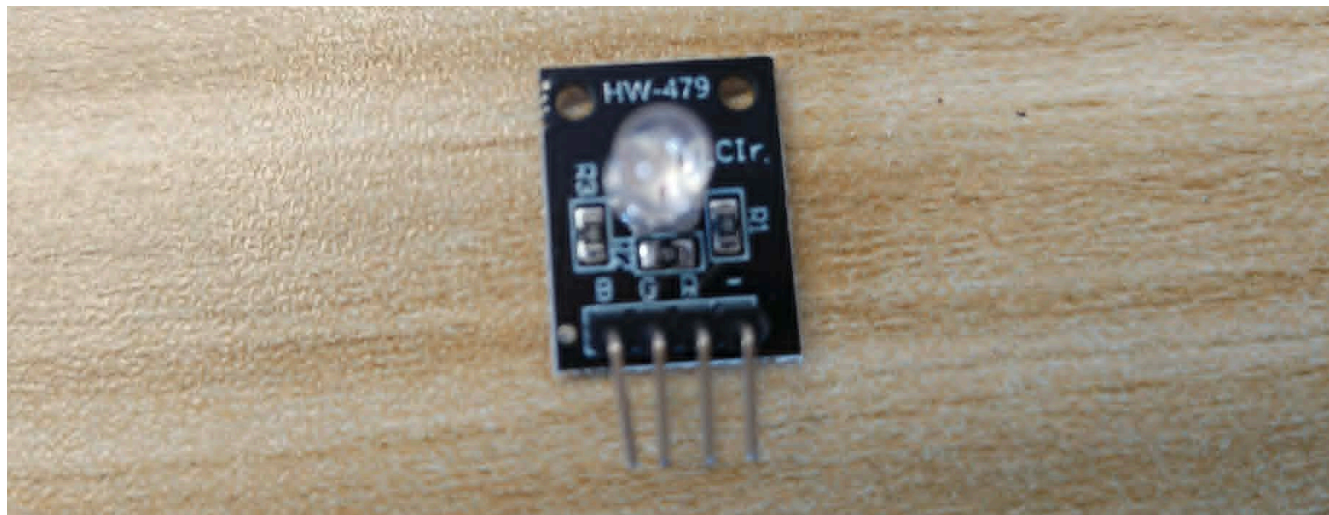
功能	T01 既有紀錄	本組確認值
RGB 紅 / 綠 / 藍	GPIO15 / 16 / 17	_ _ / _ _ / _ _
RGB 共同端、亮燈準位	– 接 GND ; HIGH 亮	_ _ / _ _
OLED 資料 / 時脈	GPIO8 / 9	_ _ / _ _
OLED 電源、位址	3V3 / GND ; 0x3C 有回應	_ _ / _ _
OLED 驅動選項	1315 有基本顯示回報	_ _
最後的光敏輸入	GPIO4，仍須本次基準	_ _

GPIO4 與 GPIO15 不同；麵包板的**第 15 列**也不是 GPIO15。之後每次遇到設定名稱，就查自己這張表，不臨時猜數字。

照片與既有回報尚未補足各色電流及 OLED 拉高電壓。教師先完成課前查核，課堂不安排每組排隊等核准；未確認項目保持斷電。

同一個外殼，三種顏色

R 是紅，G 是綠，B 是藍。



已購 HW-479 實物。圖中標示 B、G、R、-；實際接線仍核對手上那片板。

我們要控制三個不同的發光通道，不是替模組接三條電源。

沿著實物標示找出 R、G、B、共同端。不要用杜邦線顏色決定功能，也不要把它和 WS2812 燈條的接法混用。

第一個目標很小：在電腦輸入 r，實物亮紅；輸入 o，實物全關。

先認零件，不先通電。板上看得到電阻，不代表任何電壓與 GPIO 電流都適用。

先讓電腦準備好

模組都先不接 ESP32，板子只接 USB。

1. 在 Arduino IDE 用「檔案 → 開啟」打開 [RGB / OLED 倒數程式](#)。 .ino 放在同名資料夾；IDE 詢問時同意建立。
2. 開啟左側**程式庫管理員**，搜尋 **U8g2**，作者 **Oliver Kraus**，安裝教材對照的 **2.36.15**。已裝相同版本不重裝。
3. RGB 測試雖還不用螢幕，這支完整程式仍需 U8g2 才能編譯。Wire 隨 ESP32 套件提供，不另找同名程式庫。
4. YD / CH343 板的 USB 線接背面標 **COM** 的接頭。依這片板的既有設定選 **ESP32S3 Dev Module**、實際連接埠，UART 路徑使用 **USB CDC On Boot: Disabled**。
5. 用左上勾號 **Verify** 檢查編譯。右箭頭 **Upload** 才是上傳。

程式原始值 `LESSON_STAGE=0` 會阻擋輸出，下一頁才改成第一輪。這不是程式壞掉。

COM 接頭是板上的插孔；COM8 等名稱是電腦的埠號。每台電腦可能不同，不照抄教師號碼。

先選 1，只填 RGB 設定

在程式開頭搜尋	填入內容
LESSON_STAGE	1
DEVICE_ID	例如 "group01"，保留雙引號、不填姓名
PIN_RGB_R、PIN_RGB_G、PIN_RGB_B	本組三個已確認 GPIO
RGB_ON_LEVEL	已確認的 HIGH 或 LOW；不是大家都填 HIGH
RGB_PROFILE_CONFIRMED	接法及電氣條件確認完成後才 true
OLED、光敏相關設定	保持預設 false / -1， LIGHT_INTERFERENCE_TEST=false

1. 外部模組仍不接，只有裸板 USB。
2. 按 Verify，再按 Upload，等待完成。
3. 關閉 Serial Monitor，拔 USB、PWR 熄滅，再照下一頁接 RGB。

-1 是「還沒有指定可用腳位」，不是負一號排針。true 是記錄已完成確認，不是略過檢查的開關。

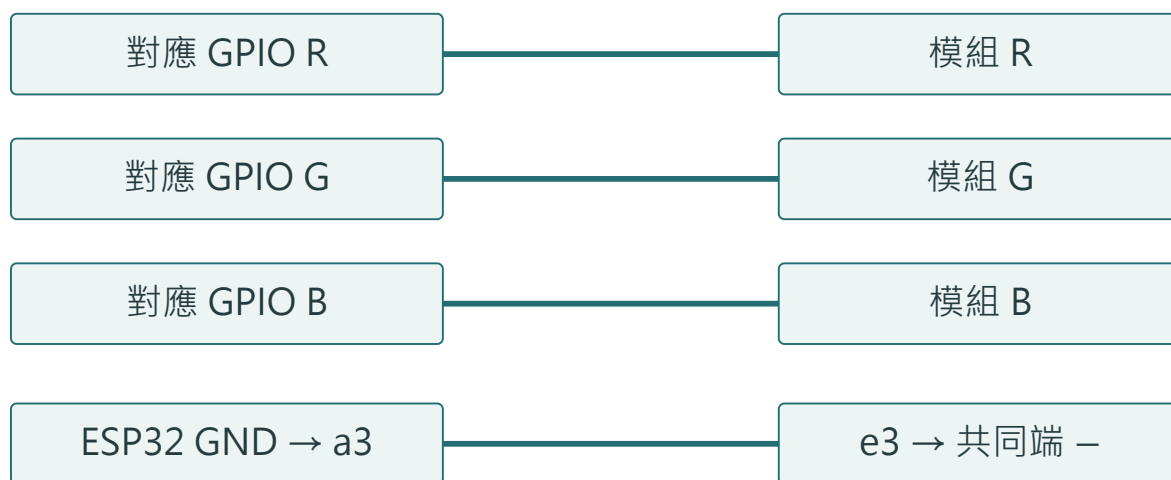
若看到 U8g2lib.h 找不到，回程式庫管理員檢查安裝，先不要改接線。

每種顏色有自己的 GPIO

以下示範已確認共同端接 GND 的模組。

USB 拔除，OLED 還沒接。依第 3 頁本組表接線：

起點	終點	線型
ESP32 GND	a3	公對母
RGB 已確認共同端 —	e3	公對母
本組紅色 GPIO	模組 R	母對母
本組綠色 GPIO	模組 G	母對母
本組藍色 GPIO	模組 B	母對母



此圖按功能排列，不代表模組排針左右順序。

雖然模組有四腳，另需一條 GND 到麵包板的線，所以這張表共五條。a3 ~ e3 內部相通，R / G / B 不放在同一五孔列。

本批共同端與限流路徑確認後才用這張圖。不同的共陽模組不能把共同端任意改接 GND；不靠換電源試錯。

依序輸入 o、r、g、b、o

每次只送一個小寫字母。

接線核對後接回 USB，開「工具 → 序列埠監控視窗」，選 **115200**。在輸入框打字後按 Enter。

送出	應看到	自己觀察到
o	全關	--
r	紅	--
g	綠	--
b	藍	--
o	全關	--

以下是**示例紀錄**，不是實物量測：

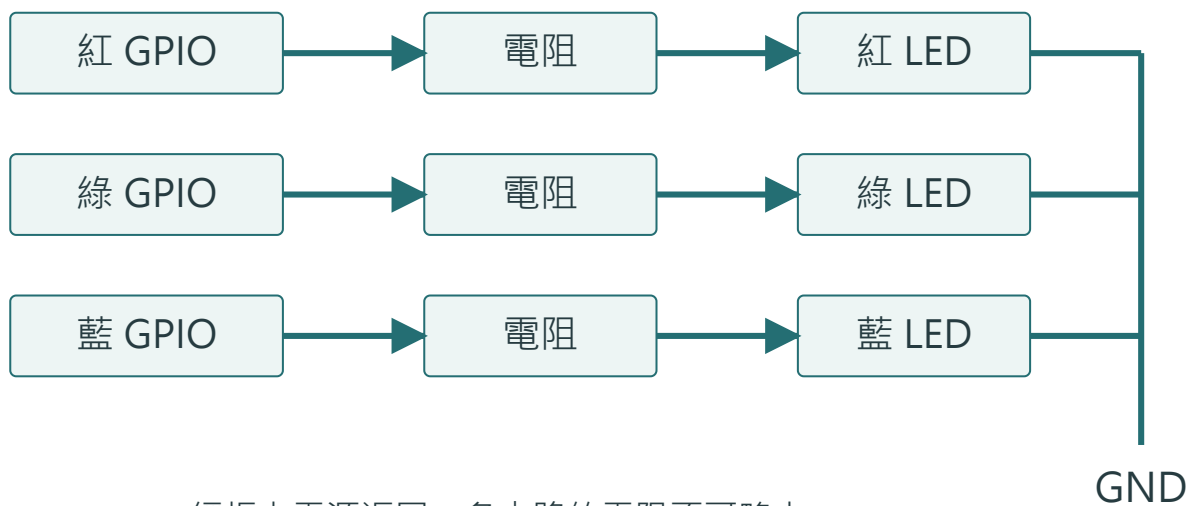
```
event_type=rgb_test requested=g visual_verified=false
```

`requested=g` 是程式要求綠；`visual_verified=false` 表示程式沒有看見實際顏色。把目視結果另外記下，不把原始紀錄改成 `true`。

送 r 卻亮綠，就拔 USB 後查 R / G 路徑。不要只把程式文字改成綠，掩蓋錯接。

燈亮，是因為形成了電流回路

這張是已確認共陰模式的簡化圖，不是另一張排針順序圖。



經板上電源返回；各支路的電阻不可略去。

三條支路各有限流電阻，共同端回到 GND。只開紅色時，另兩個通道關閉。本週不做混色或把三個控制腳直接相連。

計算示例，非本批電流實測： 假設供應 3.3 V、某一色 LED 壓降 2.0 V、串聯電阻 330 Ω 。

$$\text{電阻兩端電壓} = 3.3 - 2.0 = 1.3 \text{ V}$$

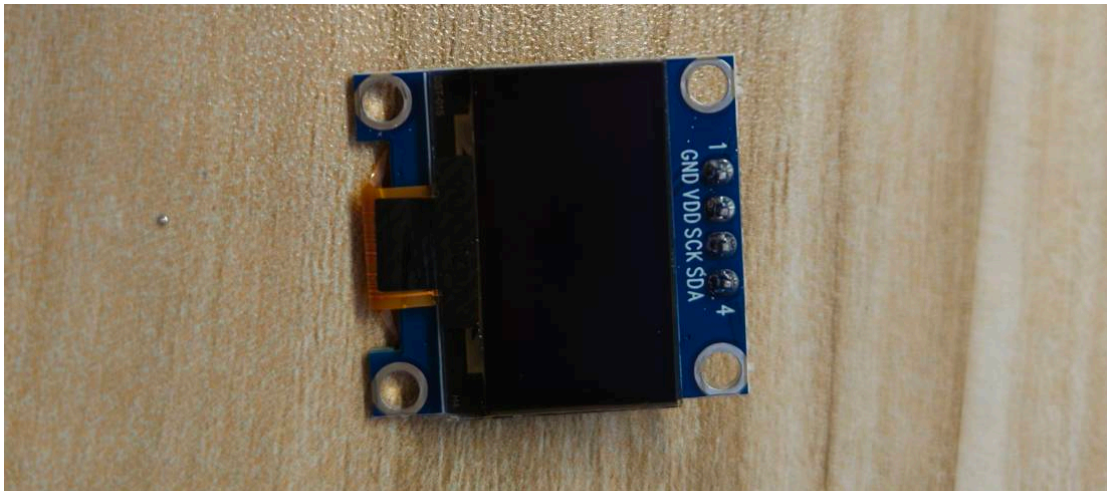
$$\text{電流} = 1.3 \div 330 \approx 0.00394 \text{ A} = 3.94 \text{ mA}$$

不同顏色的 LED 壓降可能不同。板上電阻、GPIO 單腳與總電流仍需核對，不能把這個算例當成任一模組的規格。

共陰模式通常 HIGH 亮；共陽模式可能 LOW 才亮。這就是程式要另存 RGB_ON_LEVEL 的原因。

OLED 的四腳，各做不同的事

先拔 USB，暫時拆開 RGB 和 ESP32 的連線。



已購四針 OLED 正面標示 GND、VDD、SCK、SDA。這是辨識照片，不是通電畫面。

這片板上的字	本課功能
GND	共同參考與電源回路
VDD	供電；其他模組可能寫 VCC
SCK	這片已核對 I2C 模組的時脈，程式寫 SCL
SDA	傳送資料

別的模組寫 SCK，不一定代表 I2C。七針 SPI 螢幕也不能套用本週四針圖。

安裝支架要留出背面套杜邦線的空間；不要硬推排針或把螺帽鎖到板子彎曲。

先上傳掃描程式

掃描只問「這個位址有沒有回應」，還不顯示文字。

1. RGB、OLED 都先不接 ESP32。
2. 用「檔案 → 開啟」打開 [I2C 掃描程式](#)。這是另一支程式，不貼進剛才的程式裡。
3. 在開頭填入本組 PIN_SDA、PIN_SCL，兩者必須不同。
4. 已完成本批電源、腳序與訊號電位核對後，ELECTRICAL_PROFILE_CONFIRMED 才改 true。
5. 裸板接 USB，Verify、Upload；成功後拔 USB，再接下一頁。

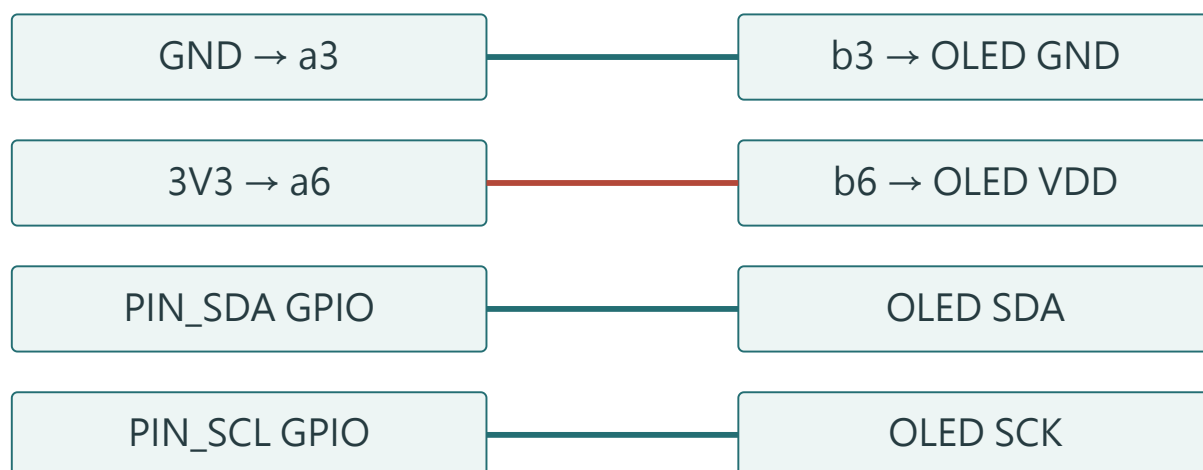
設定例的來源	不能推論的事情
T01 曾使用 SDA8、SCL9	所有 ESP32 都應接 8 / 9
曾掃到 0x3C	任一 OLED 都是 0x3C
程式上傳成功	OLED 已經能顯示

0x 是十六進位數字的寫法，這裡表示通訊位址，不是腳位編號。

兩條供電線，兩條通訊線

USB 拔除。模組已確認適用 3.3 V 供電，SDA / SCL 電位也須相容。

起點	終點	線型
ESP32 GND / 3V3	a3 / a6	各一條公對母
OLED GND / VDD	b3 / b6	各一條公對母
OLED SDA	本組 PIN_SDA GPIO	母對母
OLED SCK	本組 PIN_SCL GPIO	母對母



供電分成兩列；訊號直接連兩個不同 GPIO。

這次不用 5V_{in}。模組宣稱可接 5 V 電源，也不代表訊號可安全接 ESP32；不可自行改高電壓。

逐條追線：供電不能和訊號互換，SDA / SCL 不共用一個 GPIO。確認無錯接再接 USB。

掃到 0x3C，不等於認出型號

開 Serial Monitor，選 115200。

每次開機只掃一次。若錯過紀錄，在接法不變時按 RST 重看。

教學示例，非這次實測：

```
event_type=i2c_ack address_7bit=0x3C model=unknown
event_type=scan_finished found=1 model_verified=false
```

看見什麼	怎麼判斷
符合模組資料的位址有回應	保存完整輸出，下一步才驗證畫面
found=0	沒掃到；拔 USB 後查電源、共地及兩條訊號
多個或不符預期的位址	停下核對，不任挑常見數字
blocked	先查設定；程式沒有開始掃描

ACK 是對方的確認回應，像點名時回答「有」。它不告訴你控制器型號，也不證明像素、玻璃或文字畫面正常。

found=1 是一個有回應的位址，不必然代表恰好一片正常螢幕。

螢幕怎麼收到文字？

供電讓它工作，通訊讓它知道要畫什麼。



上：資訊與時序；下：供電與共同參考。

ESP32 在這條 I2C 通訊上主動開始傳送；SDA 傳資料，SCL 配合時脈。模組回應自己的位址。

螢幕上的控制器再把收到的內容轉成像素。**128 × 64** 是橫向 128、縱向 64 個像素，不是電壓。

所以接上電源不會自動出現「Hello」。要有正確介面、位址、控制器設定與程式送出的畫面。

同一位址可能用在不同控制器上，SSD1306、SSD1315、SH1106 不能只靠掃描結果混用設定。

第二輪只顯示文字

回到 RGB / OLED 倒數程式，改選 2。

設定名稱	本次內容
LESSON_STAGE	2
PIN_SDA、PIN_SCL	剛才已確認的 GPIO
OLED_ADDRESS_7BIT	剛才回應且符合資料的位址，例如 0x3C；不要先乘 2
OLED_CONTROLLER	1306 或 1315，依本組已確認的驅動配置
OLED_PROFILE_CONFIRMED	完成電氣與適用配置核對後才 true
RGB、光敏設定	此輪不啟動 RGB；光敏測試仍 false

T01 的 1315 曾有顯示回報，是相容配置證據，不是掃描器鑑定出晶片。若是 SH1106、SPI 或其他解析度，不在目前兩個選項內，先不硬套。

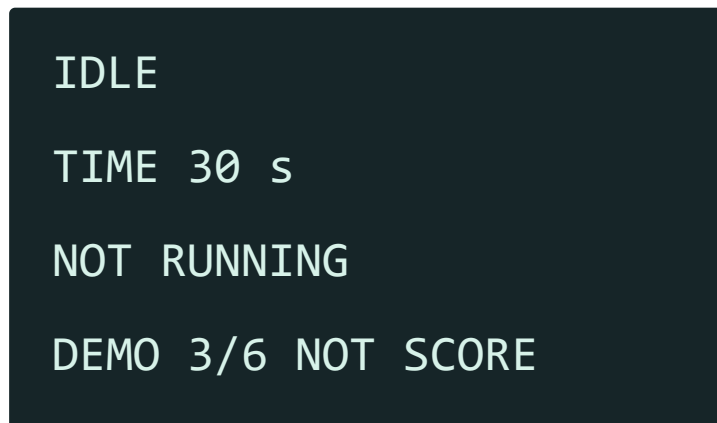
1. 拔 USB，先拆開 OLED 與 ESP32 的連線。
2. 裸板接 USB，Verify、Upload；完成後拔 USB。
3. 依第 12 頁恢復 OLED 接線，核對後上電。RGB 仍不接。

完整程式仍需 U8g2。若安裝過同版本，不必再次安裝。

先讓 3 變 4，再回到 3

這一輪還不能開始倒數。

上電後應看到以下畫面示意，非實機截圖：



版面示意，不是實機照片；四行都要能看清楚。

1. 確認四行字都完整、沒有偏移或缺行。
2. 在 Monitor 送 4，看 DEMO 變成 4/6。
3. 再送 3，看它回到 3/6。
4. 送 s 應被拒絕，因為現在只做第 2 輪。

IDLE 是待機；TIME 30 s 是準備的總時長，還沒開始走。NOT RUNNING 是未進行中。NOT SCORE 提醒這個數字**不是實際得分**。

如果畫面不完整，保存照片後拔 USB，核對驅動、解析度與位址。ACK 有回應仍可能畫錯。

程式先畫在記憶體，再送到螢幕



`clearBuffer()` 清掉的是 ESP32 記憶體裡準備傳送的影像，不是切斷 OLED 電源。

`sendBuffer()` 才把這張圖送出去。

這支程式每次重畫整張，避免把新的短字疊在舊的長字上。傳送要花時間，畫面是某個時間點的結果，可能比目前狀態稍舊。

128 × 64 的單色像素緩衝需要：

$128 \times 64 \div 8 = 1024 \text{ bytes}$
每個像素一個 bit ; 8 bits = 1 byte

這只算像素資料，不是整支程式的全部 RAM。螢幕先使用英文字與數字；中文字另需適用字型與實機排版，不直接換中文字就當完成。

第三輪把燈和螢幕合起來

RGB 三色與 OLED 固定文字都正常後，再做整合。

1. USB 拔除，先取下模組與 ESP32 間的連線。
2. 主程式改 **LESSON_STAGE = 3**，保留兩邊已確認的設定與 true；光敏測試仍 false。
3. 確認 R、G、B、SDA、SCL 五個 GPIO 不重複。裸板 Verify、Upload，完成後拔 USB。
4. 依下表接回，查線後上電。應是 **IDLE、TIME 30 s、RGB 全關**。

連接	本次位置
ESP32 GND / 3V3	a3 / a6
OLED GND / VDD	b3 / b6
RGB 共同端	已確認共陰配置接 e3
RGB R / G / B	各自原來的 GPIO
OLED SDA / SCK	各自原來的 GPIO

以下為已確認共陰 RGB 的配置



R、G、B、SDA、SCL → 五個不同 GPIO

不要把「共用 GND」誤解成所有訊號接同一列。

送 s，等它自己到期

先不按其他命令，觀察一個完整的 30 秒。

開始後的經過時間	程式預期狀態與燈色
0 ~ 不到 3 秒	RUNNING；綠
3 ~ 不到 6 秒	RUNNING；紅
6 ~ 不到 9 秒	RUNNING；綠
此後每 3 秒	紅綠輪換
檢查到經過至少 30 秒	EXPIRED；全關；剩餘 0

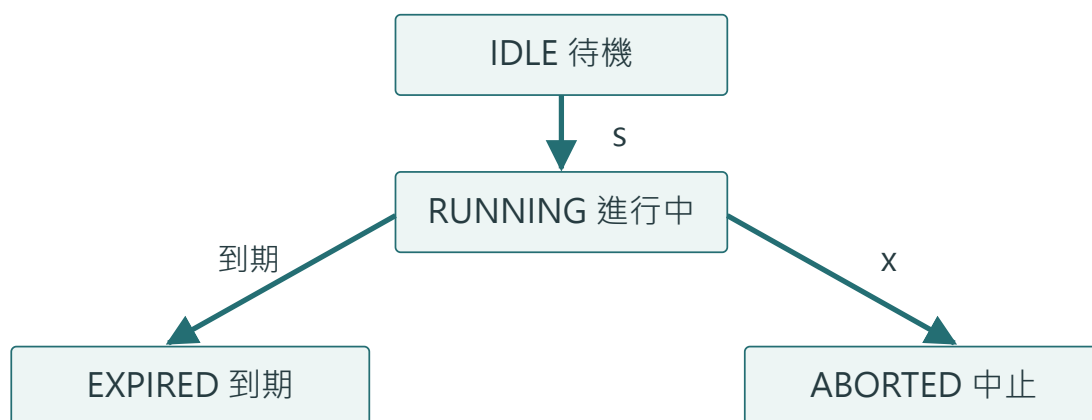
同時對照 **實物燈**、**OLED 文字**、**Serial 的 phase**。GREEN / COVER 是綠燈提示，RED / WAIT 是紅燈提示；這週遮住 KY 也不會因此得分。

到期後再送 s，程式會拒絕。送 z 回待機，再送新的 s，才開始下一輪。

保存一輪的開始、換色與到期紀錄，以及實物照片。不要把每一次螢幕刷新當成一秒。

x 是中止，z 是重新準備

1. z 回 IDLE，s 開始新一輪。
2. 進行中再送 s，確認它不重設起點。
3. 送 x：應變 ABORTED、RGB 全關，剩餘時間停止變化。
4. 再送 s，應被拒絕。
5. 送 z，回到 IDLE、30 秒；還不計時。再送 s 才重新開始。



兩者都先 z 回待機，再 s 才開始新一輪。

存在顯示故障時，z 仍被拒絕。

中止不是暫停後續跑。z 會準備新一輪，不接著剩下的秒數走。

x 是軟體命令，不是切斷電源。發熱、異味、持續重啟或需要換線時，直接拔 USB，不靠送 x 代替斷電。

30 秒不是「畫面減 30 次」

先看開始時刻，再看現在時刻。

以下為**教學假資料**：開始時 `millis()` 是 10000，現在是 11500。`millis()` 表示開機以來的毫秒。



兩個時刻相減 → 經過 1500 ms

時間軸示意，非等比例。

```
經過時間 = 11500 - 10000 = 1500 ms  
剩餘時間 = 30000 - 1500 = 28500 ms = 28.5 s
```

開始時刻、目前時刻是時鐘上的位置；經過時間和剩餘時間是長度。1 秒 = 1000 毫秒。

程式每次用時間差重算，就算畫面少更新幾次，也不會因此補回幾秒。若已過 30 秒，剩餘值為 0，不顯示負數。

還有 1 毫秒，畫面也可能寫 1 秒

畫面整秒方便讀，判定到期仍使用毫秒。

remaining_ms	顯示 seconds	真正意思
28500	29	還有 28.5 秒，不是完整 29 秒
1499	2	還有 1.499 秒
1000	1	還有 1 秒
1	1	只剩 0.001 秒
0	0	已到期

顯示秒數 = (剩餘毫秒 + 999) 整數除以 1000

這是向上取整。例如 $(1499 + 999) \div 1000$ ，取整數結果為 2。

畫面還可能是上一張舊圖。因此，不能拿螢幕仍顯示 1，主張時鐘已到期後還能繼續。

若直到經過 31200 ms 才再次檢查，仍應到期、剩 0；但已晚了 1200 ms，不能宣稱精準在第 30 秒停止。

程式的狀態，和你看到的畫面

以下示例分行方便閱讀，不是新的實測結果。

```
event_type=timer_status state=RUNNING
remaining_ms=28500 seconds=29 phase=GREEN
io_ok=true reason=serial_start uptime_ms=11500
```

欄位	判讀
state	RUNNING：程式認定這輪正在進行
remaining_ms / seconds	同一剩餘時間的毫秒與整秒表示
phase	程式此時的紅綠規則，不是光學量測
io_ok	沒記到此程式可偵測的顯示故障，不等於所有硬體都健康
reason	最近狀態原因，不表示每一行重新開始
uptime_ms	開機後時刻，不是日期、剩餘時間或埠號

另一行 `view_snapshot` 的 `demo_count` 是人工示例，`rgb_requested` 是要求的燈色。
`visual_verified=false` 仍需你觀察實物。

選取完整輸出，複製到課堂筆記再存檔。Serial Monitor 不會自動替你保存紀錄。

只把刷新間隔 200 改成 1500

保持總時長 30000、換色間隔 3000 不變。

先預測：螢幕較慢刷新，這輪會不會變成 225 秒？會不會跳秒？

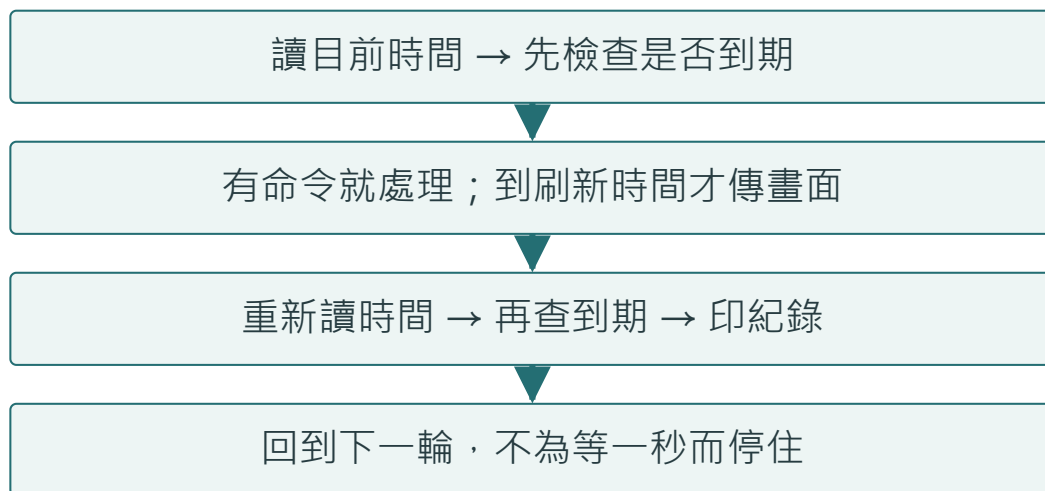
1. 先保存原設定 200 ms 那一輪的紀錄。
2. 只改 `DISPLAY_INTERVAL_MS = 1500`，Verify、Upload；接線不變，不移線。
3. 送 s，保存 `remaining_ms`、`uptime_ms` 與到期結果，觀察畫面更新。
4. 比較兩版本，完成後改回 200，重新上傳、確認正常。

保持不變	唯一改變
同組板、模組、接線、30 秒、3 秒換色	畫面更新間隔 200 → 1500 ms

預期畫面較常跳秒或落後，**不應把 30 秒乘 7.5 變 225 秒**。剩餘量仍由時間差計算。

這是刻意改程式，不是硬體突然壞了。紀錄能比較軟體判定；沒有外部時間參考，不能宣稱已驗證時鐘的絕對精度。

程式不必為了等一秒停住



本例在倒數迴圈用時間差判斷，不用 `delay(1000)` 讓整個迴圈停一秒。開機時的 `delay` 是另一件事，不是倒數算法。

畫面約每 200 ms 更新；一般狀態紀錄約每 500 ms 輸出，狀態改變時也會印。這兩個間隔都不是「一秒」的定義。

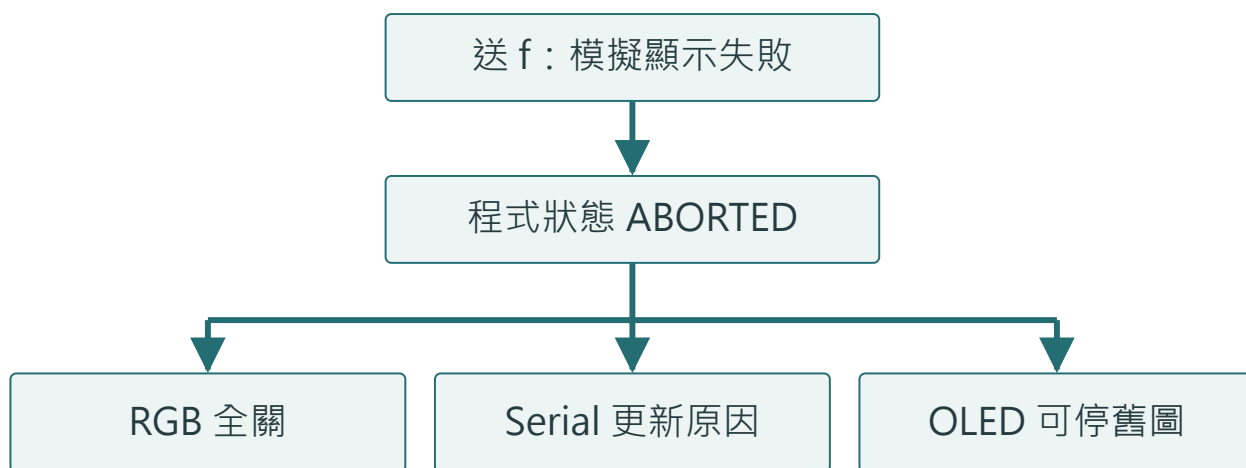
但 I2C 傳一張圖仍要花時間。程式傳完重新讀時鐘、再次判斷到期，不能保證零延遲。

程式的 20 ms 匯流排逾時設定，也不是整張圖、整個中止流程一定在 20 ms 內完成的保證。

送 f，不要拔帶電訊號線

保持原接線，讓軟體走一次故障處理。

1. 確認回到原本 200 ms 刷新設定。
2. z 回待機，s 開始；進行中送 f。
3. 查 Serial：應為 **ABORTED**、io_ok=false，原因 injected_display_failure。
4. 看 RGB 是否全關。OLED 可能仍停在舊畫面，因為程式刻意停止傳送。
5. 送 z，應被拒絕；不能略過故障直接重開。



三者要分開觀察，不只看螢幕。

畫面還寫 RUNNING，不代表程式仍在計時。先對照 Serial 的狀態、原因與實際燈號。

這是軟體故障注入，不是真正拔線測試。通過只支持這條錯誤處理流程，不代表所有螢幕故障都能被偵測。

c 只重新檢查，不自動續跑

操作	應看到
送 c	取消注入、重新詢問實際 OLED 的 ACK
ACK 成功	原因為 <code>ack_recovered_reset_required</code> ，仍 ABORTED
等畫面更新、目視完整	確認狀態、文字與實物不矛盾
再送 z	回 IDLE、TIME 30 s，還沒開始
再送 s	一輪新的倒數

若 c 之後是 `display_still_unresponsive`，維持中止、拔 USB 後查線，不一直按 z 硬開。

即使 ACK 恢復，玻璃破裂、缺像素或畫面錯亂也可能還在。必須目視，不能用一個 `true` 當作全部恢復。

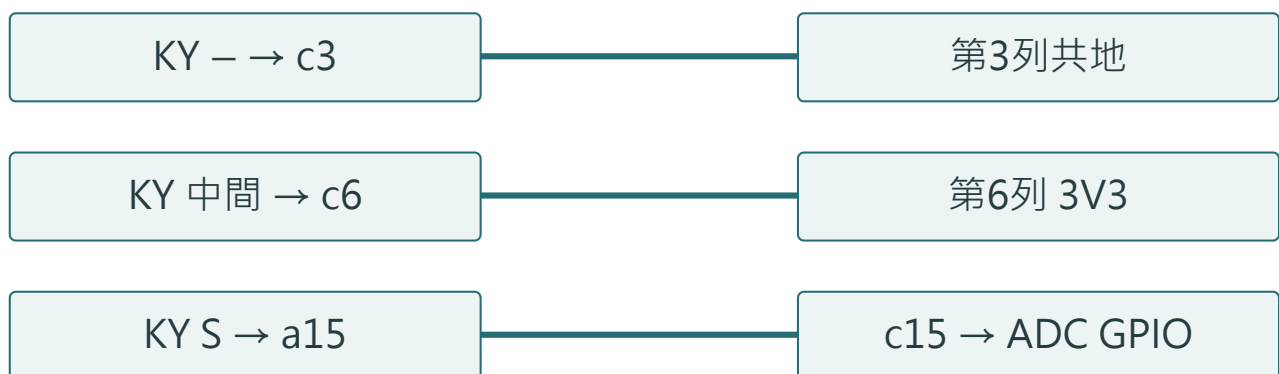
記下 f、z 被拒絕、c 結果、目視恢復與新一輪。不要刪除失敗紀錄只留最後成功。

確認自己的燈不會誤導感測器

這次量干擾，不計分，也不接 DHT11。

USB 拔除。保留 RGB / OLED 功能接法，新增：

起點	終點
KY 已確認的 - / 中間電源腳	c3 / c6
KY S	a15
本組已確認 ADC GPIO	c15 ; T01 紀錄使用 GPIO4



不是把第15列接到 RGB 的 GPIO15。

設定 `LIGHT_INTERFERENCE_TEST=true`、`PIN_LIGHT` 為本組 ADC 腳位；接法確認後 `LIGHT_PROFILE_CONFIRMED=true`。六個訊號 GPIO 不能重複，`LESSON_STAGE` 保持 3。

先取下模組至 ESP32 的連線，裸板上傳，完成後拔 USB，再依已確認表接回。上電後保持 **IDLE**，不送 s。

第 15 列的 S 訊號不能和控制 RGB 的 GPIO15 混在一起。

不遮光，只換燈色

KY 位置、方向、桌面照明與 OLED 畫面保持不變。

1. 送 **o** 全關，穩定後保存 5 筆 `interference_sample`。
2. 送 **r** 紅燈，同樣保存 5 筆。
3. 送 **g** 綠燈，同樣保存 5 筆。手不要遮住 KY。
4. 比較各組 `light_raw` 最小 / 最大，對照本組 Week 3 分界。

條件	最小 raw	最大 raw	是否跨過本組分界
全關	--	--	--
紅	--	--	--
綠	--	--	--

若會影響分類，先調整 RGB 和 KY 的距離、方向，或用不遮住手勢區的遮板隔開光路。一次只改一項，再比較同樣三種條件，不一邊換桌一邊改門檻。

沒有跨門檻，只支持這次布局。數字一變也不等於短路；它可能是光學影響，仍需排除其他原因。有發熱或重啟就先斷電，不為湊數據繼續供電。

看得到倒數，就表示它算對了嗎？

以下都是教學假資料；可以查前面講義，先寫理由再翻頁。

1. 30 秒倒數在開機時間 10000 ms 開始，現在 11500 ms。經過多久？剩幾毫秒？畫面顯示幾秒？
2. 畫面先隔 0.8 秒更新，再隔 1.3 秒更新。每次更新就把 30 減 1，得到 28。正確剩多久？兩種方法畫面都寫 28，能證明算法相同嗎？
3. 剩 1 ms 時畫面寫 1；下一次檢查已到期限，能繼續嗎？
4. OLED 顯示 GREEN、實物亮紅、Serial 的 phase=RED。下一步做什麼？單張照片能確定是誰壞了嗎？
5. f 後送 c，ACK 恢復但畫面一半不亮，可以直接送 z、s 重開嗎？
6. 玩家沒有遮光，換紅燈時 raw 跨過分界，同時畫面少一秒，能算一次遮光得分嗎？

下一頁就是解答。 每題指出你使用的數字、欄位或觀察，不只寫「可以 / 不可以」。

畫面、時間與實物要分開驗證

題目	答案與理由
1	經過 1500 ms；剩 28500 ms；向上取整顯示 29 秒。
2	經過 2.1 秒，剩 27.9 秒，取整也是 28。錯誤方法內部卻當成剩 28 秒，差 0.1 秒。一次畫面相同不能證明算法等價。
3	不能。顯示 1 包含只剩 1 ms，也可能是舊畫面；到期以時間差判定。
4	先 x 中止並保存紀錄。可能畫面落後或接線 / 文字錯，需比對時間與路徑；改線先拔 USB。不能憑單張照片確定原因。
5	不可以。ACK 不證明像素與畫面正常，先保持中止、確認顯示恢復。
6	不能。可能是 RGB 光學干擾，時間顯示改變也不是手勢證據；而本週程式根本沒有遮光計分。

刷新比較那一頁也應得到：200 → 1500 ms 只改畫面間隔，不應把 30 秒變成 225 秒；仍要保留實際紀錄，不把預期抄成實測。

把倒數器變成能直接按的作品

用 Week 2 的兩顆按鈕，控制本週 OLED。USB 仍供電，但不用鍵盤送 s / x。

零件	本例用途
第一顆按鈕	待機時調整倒數長度
第二顆按鈕	開始；進行中再按一次就中止
OLED	顯示待機、倒數、中止或到期
Serial	留下調整、開始、中止、到期與顯示故障紀錄

先做桌上十秒倒數器，再改成五秒一格。按鈕規則沿用 Week 2：長按只接受一次，放開後才能再按。

本例只保留按鈕與 OLED。RGB、KY、DHT 都先移除，避免 GPIO4 / 5 用途衝突，也不增加用電。整合不是把所有零件一次接滿。

本段可選擇取代重複操作，建議 20–30 分鐘；不新增作業分數。先完成 OLED 單獨顯示，才加按鈕。

OLED 不變，換上兩顆按鈕

拔 USB、拆掉全部外接線，開啟 [完整按鈕倒數程式](#)。填好本組確認值，只接 USB 上傳，再拔 USB 接線。

功能	接線 / 設定
共地	板 GND → a3；OLED GND → b3；兩按鈕 B 端 → c3、e3
電源	板 3V3 → a6；OLED VDD → b6
調整鍵	A 端 → PIN_ADD，圖示板確認後用 GPIO4
開始 / 中止鍵	A 端 → PIN_START，圖示板確認後用 GPIO5
OLED 訊號	SDA、SCK 各接本組已驗證的 PIN_SDA、PIN_SCL

T01 歷史設定為 SDA8、SCL9、0x3C、1315 相容配置，**不是其他 OLED 的保證**。把前段已驗證的值填入 OLED_ADDRESS 和 OLED_CONTROLLER；確認供電及訊號後才把 PROFILE_CONFIRMED 改 true。

兩顆按鈕都用 Week 2 測過的 A / B 端，放開不通、按下才通。兩條 GPIO 不得同列相接。原檔 -1 / false 會阻擋啟動。

調整、開始、中止，再開始

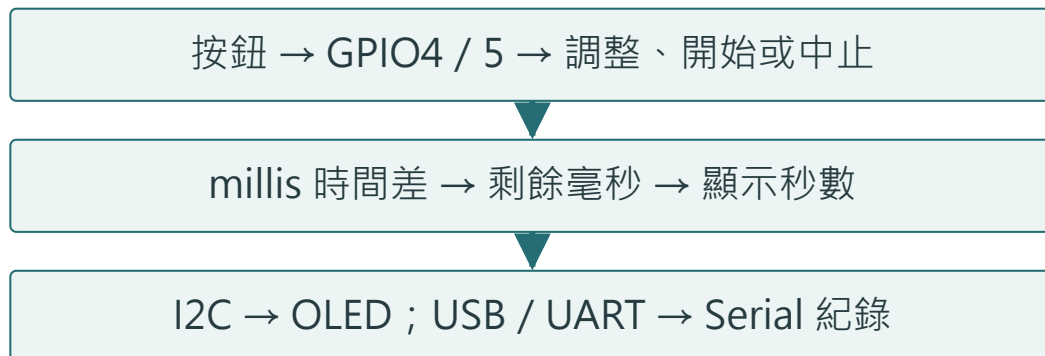
接 USB，Monitor 選 115200。畫面應為 IDLE、TIME 10 s；先放開兩鍵至少 0.1 秒。

操作	預期結果；另記自己的實測
按調整鍵一次	10 → 20 秒；按住不連加
按開始鍵	RUNNING，從 20 秒倒數
進行中按調整鍵	不改時長；log 印 ignored_while_running
再按開始鍵	STOPPED，剩餘時間凍結
放開再按開始鍵	重新從 20 秒開始，不是續跑
放著到期	DONE、0 秒；只記一次 expired
待機調至 60 秒，再調一次	回 10 秒

兩鍵同時按住不接受新動作；已先接受的單鍵動作不會撤回，和 Week 2 一樣不是精密的同時判定。

此程式沒有前段的 s / x / f / c 指令。顯示通訊失敗會停止，保持 FAULT；保存紀錄、斷電排查，恢復後 Reset 才重來。

按鈕是命令，時間仍用 millis



任一按鈕按下的電流：

3.3 V → 晶片內部上拉電阻 → 對應 GPIO → 按鈕 → GND

OLED 供電：3V3 → OLED 電路 → GND → 板上電源

OLED 資訊：ESP32 → SDA 資料 / SCL 時脈 → OLED

GPIO4 / 5 都只讀按鈕，不供應螢幕電力。SDA / SCL 是另一組不同的 GPIO。

開始時記下時間，之後用時間差計算剩餘毫秒。OLED 約每 200 ms 更新，不是每畫一次就減一秒。

ACK 仍不代表畫面一定正常。畫面和 log 不一致時先中止、保存紀錄，再排查；本例不是醫療、安全或精密計時裝置。

改成五秒一格的小計時器

沿用同一套接線，不加零件。

要求：開機 5 秒；每按調整鍵加 5 秒；最大 30 秒，再按回 5 秒。開始 / 中止、長按一次與同按處理保持不變。

1. 找到 STEP_SECONDS 與 MAX_SECONDS，先寫預測值。
2. 修改、上傳，列出連按六次調整鍵的數列。
3. 設成 15 秒開始，約五秒後中止。預測再次開始是約 10 秒，還是完整 15 秒？
4. 進行中按調整鍵，確認倒數設定沒有改變。
5. 保留 adjust、start、stop、expired 的實際 log。畫面也要目視確認。

請先做再翻頁。下一頁就是程式設定與測試解答。

改兩個設定，保留操作規則

```
const uint32_t STEP_SECONDS = 5;  
const uint32_t MAX_SECONDS = 30;
```

取代完整程式中的同名設定，其他程式不改。起始值本來就取自 STEP_SECONDS，因此不另找一個 10 改掉。

測試	應有結果
開機	IDLE、5 秒
連按六次調整鍵	10、15、20、25、30、5 秒
設成 15 秒，中止後再開始	重新從完整 15 秒開始；不是暫停續跑
進行中按調整鍵	ignored_while_running；設定維持 15 秒
長按任一鍵	一次操作，須放開才能再接受
到期後持續等待	維持 DONE、0 秒，不一直重印 expired

這是手動操作的作品，**不代表自動偵測或雲端 IoT 已完成**。本週先做好硬體操作與 log，後續再讓手機或伺服器使用相同操作規則。

留下能對照的三種證據

軟體紀錄、畫面照片、實物觀察，放在原本的課堂筆記即可。

保存的內容	要能回答
本組腳位與設定、接線照片	同一組器材能否照記錄重接？
RGB 五個命令、OLED 掃描與 4→3	程式要求和實物是否一致？
一輪到期、一輪中止及重開	s、x、z 分別做什麼？
200 / 1500 ms 比較、f / c 恢復	有沒有混淆刷新、時間與故障？
光敏三種燈色各 5 筆、布局照片	有沒有控制其他條件？有哪些限制？
練習題的理由與修正	能否解釋，不只照抄成功畫面？

保存完整文字與程式設定，不只截最後一行。沒有做實物的部分就寫未實測；沒有額外新增報告或分數。

收尾先送 x。正常且無故障時可 z 回待機、o 全關；任何異常直接拔 USB。存好紀錄後拔 USB、確認 PWR 熄滅、電表 OFF，再拆線。

下一週可沿用已確認的顯示與狀態，增加其他輸出。這週的成功不代表舵機或外部供電已完成測試。

一次只查一件事

情況	下一個檢查
U8g2lib.h 找不到	查 Oliver Kraus 的 U8g2 是否安裝
profile_missing	查輪次、必要設定、重複 GPIO；不全部改 true
Monitor 沒連上	查 COM 接頭、實際埠號、Board、115200；不先改 OLED 線
r 卻亮綠	拔 USB 後沿 R / G 逐條查，不修改紀錄掩蓋
OLED 掃不到	拔 USB 後查電源、GND、SDA、SCL，不提高電壓
有 ACK 但畫面亂	保存照片，查控制器、解析度、驅動與位址格式
s 被拒絕	必須第 3 輪、IDLE、無顯示故障；不是連按 s
f 後螢幕不動	先看 Serial 的 ABORTED，這可能就是舊畫面
發熱、異味、重啟	立即拔 USB，停止本次實驗再排查

求助時附上：程式設定、完整訊息、接線照片、最後一個命令。不要只說「螢幕不能用」。

不用先看懂每一行

要找的行為	程式位置
哪些器材可以啟動	開頭設定、 <code>profileReady()</code>
剩餘時間	<code>remainingAt()</code> ，從開始時刻算差
顯示整秒	<code>displaySeconds()</code> ，向上取整
換色與到期	<code>advanceTimer()</code> ，先判期限
s、x、z、f、c	<code>handleCommand()</code> ，依狀態處理
四行螢幕內容	<code>drawSnapshot()</code>
不斷重複檢查	<code>loop()</code> ，傳圖前後都檢查時間

Wire 掃描用 7 位址，例如 0x3C。主程式傳給 U8g2 時才乘 2，這是 API 格式要求，**不是掃到了第二個位址**。設定表仍填原來的 0x3C。

Verify 的 Sketch uses 比例對照的是設定的應用程式分割區，不是整片 16 MB Flash；Global variables use 也不是執行時所有 RAM、堆疊與 PSRAM 用量。

這些編譯數字不能證明接線或畫面正常。後面兩支完整程式保留未確認時的阻擋設定，依課堂設定再使用。

官方資料與完整程式

查什麼	來源
SDA / SCL、Wire 與逾時	Espressif Arduino-ESP32 I2C
OLED 控制器與驅動選項	U8g2 建構子說明
位址、清緩衝區、傳畫面	U8g2 API 參考
用時間差安排工作	Arduino Blink Without Delay
本批 YD 板型與接頭	YD-ESP32-S3 廠商文件

完整程式依序是 **I2C 掃描器**、**RGB / OLED 倒數器**。每支跨頁程式要合起來才完整，建議開啟本週提供的原始檔，不從 PDF 複製換行後的程式。

圖片中的接線功能與畫面示意，不代替實物核對。文件、編譯、電腦替代測試與硬體通過，是不同的證據。

week05_i2c_check

第 1 ~ 27 行。所有分頁合起來才是完整程式。

```
#include <Wire.h>

// Confirm the exact module's power, logic voltage, pin labels and bus pins first.
const bool ELECTRICAL_PROFILE_CONFIRMED = false;
const int PIN_SDA = -1;
const int PIN_SCL = -1;
bool scanned = false;

void setup() {
  Serial.begin(115200);
  delay(1000); // Startup visibility only; this is not a countdown.
  if (!ELECTRICAL_PROFILE_CONFIRMED || PIN_SDA < 0 || PIN_SCL < 0 ||
      PIN_SDA == PIN_SCL) {
    Serial.println("week=5 status=blocked reason=i2c_electrical_profile_missing");
    return;
  }
  if (!Wire.begin(PIN_SDA, PIN_SCL, 100000)) {
    Serial.println("week=5 status=blocked reason=i2c_begin_failed");
    return;
  }
  Wire.setTimeout(20);
  int found = 0;
  for (int address = 8; address <= 119; ++address) {
    Wire.beginTransmission(address);
    const uint8_t error = Wire.endTransmission();
    if (error == 0) {
      ++found;
    }
  }
}
```

下一頁接續同一支程式，不另開草稿。

week05_i2c_check

第 28 ~ 37 行。所有分頁合起來才是完整程式。

```
        Serial.printf("event_type=i2c_ack address_7bit=0x%02X model=unknown\n", address);
    }
}
scanned = true;
Serial.printf("event_type=scan_finished found=%d model_verified=false\n", found);
}

void loop() {
    // One scan per boot; never turn ACK into proof of controller or correct wiring.
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。

week05_rgb_oled_timer

第 1 ~ 30 行。所有分頁合起來才是完整程式。

```
#include <Wire.h>
#include <U8g2lib.h>

// 1: RGB only; 2: OLED only; 3: combined countdown. All start disabled.
const int LESSON_STAGE = 0;
const char* DEVICE_ID = "BOARD-T01";
const bool RGB_PROFILE_CONFIRMED = false;
const bool OLED_PROFILE_CONFIRMED = false;
const int PIN_RGB_R = -1;
const int PIN_RGB_G = -1;
const int PIN_RGB_B = -1;
const int RGB_ON_LEVEL = -1;
const int PIN_SDA = -1;
const int PIN_SCL = -1;
const int OLED_ADDRESS_7BIT = -1;
const bool LIGHT_INTERFERENCE_TEST = false;
const bool LIGHT_PROFILE_CONFIRMED = false;
const int PIN_LIGHT = -1;
// This constructor is ONLY for an approved SSD1306 128x64 I2C module.
// Other controllers/resolutions need a matching reviewed constructor, not a guess.
// ACK is not a controller ID. Select after electrical and fixed-screen checks.
#ifndef OLED_CONTROLLER
#define OLED_CONTROLLER 1306
#endif
#if OLED_CONTROLLER == 1306
U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE, PIN_SCL, PIN_SDA);
#elif OLED_CONTROLLER == 1315
U8G2_SSD1315_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE, PIN_SCL, PIN_SDA);
#else
#error Unsupported OLED_CONTROLLER: use a verified 1306 or 1315 configuration.
```

下一頁接續同一支程式，不另開草稿。

week05_rgb_oled_timer

第 31 ~ 60 行。所有分頁合起來才是完整程式。

```
#endif
const uint32_t DURATION_MS = 30000;
const uint32_t PHASE_MS = 3000;
const uint32_t DISPLAY_INTERVAL_MS = 200;
const uint32_t LOG_INTERVAL_MS = 500;
enum TimerState { IDLE, RUNNING, EXPIRED, ABORTED };
TimerState state = IDLE;
bool ready = false, ioFault = false, injectedDisplayFault = false;
bool changed = true, greenPhase = false;
int demoCount = 3;
const char* rgbRequested = "OFF";
uint32_t startedAt = 0, frozenRemaining = DURATION_MS, lastDraw = 0, lastLog = 0;
const char* reason = "boot";

bool hasRgb() { return LESSON_STAGE == 1 || LESSON_STAGE == 3; }
bool hasOled() { return LESSON_STAGE == 2 || LESSON_STAGE == 3; }
bool profileReady() {
    if (LESSON_STAGE < 1 || LESSON_STAGE > 3) return false;
    int pins[6], count = 0;
    if (hasRgb()) {
        if (!RGB_PROFILE_CONFIRMED || (RGB_ON_LEVEL != LOW && RGB_ON_LEVEL != HIGH)) return false;
        pins[count++] = PIN_RGB_R; pins[count++] = PIN_RGB_G; pins[count++] = PIN_RGB_B;
    }
    if (hasOled()) {
        if (!OLED_PROFILE_CONFIRMED || OLED_ADDRESS_7BIT < 8 || OLED_ADDRESS_7BIT > 119)
            return false;
        pins[count++] = PIN_SDA; pins[count++] = PIN_SCL;
    }
    if (LIGHT_INTERFERENCE_TEST) {
        if (!LIGHT_PROFILE_CONFIRMED) return false;
        pins[count++] = PIN_LIGHT;
    }
}
```

下一頁接續同一支程式，不另開草稿。

week05_rgb_oled_timer

第 61 ~ 92 行。所有分頁合起來才是完整程式。

```
}
for (int i = 0; i < count; ++i) {
    if (pins[i] < 0) return false;
    for (int j = 0; j < i; ++j) if (pins[i] == pins[j]) return false;
}
return true;
}

const char* stateName() {
    switch (state) {
        case IDLE: return "IDLE";
        case RUNNING: return "RUNNING";
        case EXPIRED: return "EXPIRED";
        default: return "ABORTED";
    }
}

uint32_t remainingAt(uint32_t now) {
    if (state != RUNNING) return frozenRemaining;
    const uint32_t elapsed = now - startedAt; // Unsigned subtraction handles one wrap.
    return elapsed >= DURATION_MS ? 0 : DURATION_MS - elapsed;
}

uint32_t displaySeconds(uint32_t remaining) { return (remaining + 999u) / 1000u; }

void rgb(bool r, bool g, bool b) {
    if (!hasRgb() || !ready) return;
    rgbRequested = r ? "RED" : g ? "GREEN" : b ? "BLUE" : "OFF";
    const int off = RGB_ON_LEVEL == HIGH ? LOW : HIGH;
    digitalWrite(PIN_RGB_R, r ? RGB_ON_LEVEL : off);
    digitalWrite(PIN_RGB_G, g ? RGB_ON_LEVEL : off);
    digitalWrite(PIN_RGB_B, b ? RGB_ON_LEVEL : off);
}

void abortTimer(uint32_t now, const char* why) {
    frozenRemaining = remainingAt(now);
    state = ABORTED; greenPhase = false; reason = why; changed = true;
}
```

下一頁接續同一支程式，不另開草稿。

week05_rgb_oled_timer

第 93 ~ 125 行。所有分頁合起來才是完整程式。

```
    rgb(false, false, false);
}
void advanceTimer(uint32_t now) {
    if (state != RUNNING) return;
    if (now - startedAt >= DURATION_MS) {
        frozenRemaining = 0; state = EXPIRED; greenPhase = false;
        reason = "deadline"; changed = true; rgb(false, false, false);
    } else {
        const bool next = ((now - startedAt) / PHASE_MS) % 2 == 0;
        if (greenPhase != next) changed = true;
        greenPhase = next; rgb(!greenPhase, greenPhase, false);
    }
}
bool oledAck() {
    if (injectedDisplayFault) return false;
    Wire.beginTransaction(OLED_ADDRESS_7BIT);
    return Wire.endTransmission() == 0;
}
void handleCommand(char c, uint32_t now) {
    if (c == '\r' || c == '\n' || c == ' ') return;
    if (c == 'x') { abortTimer(now, "manual_abort"); return; }
    if (c == 'f' && hasOled()) {
        injectedDisplayFault = true; ioFault = true;
        abortTimer(now, "injected_display_failure"); return;
    }
    if (c == 'c' && hasOled()) {
        injectedDisplayFault = false;
        // Allow a new physical ACK test, but do NOT resume the countdown.
        ioFault = !oledAck();
        reason = ioFault ? "display_still_unresponsive" : "ack_recovered_reset_required";
        changed = true; return;
    }
    if (c == 'z' && state != RUNNING && !ioFault) {
```

下一頁接續同一支程式，不另開草稿。

week05_rgb_oled_timer

第 126 ~ 152 行。所有分頁合起來才是完整程式。

```
    state = IDLE; frozenRemaining = DURATION_MS; greenPhase = false;
    reason = "manual_reset"; changed = true; rgb(false, false, false); return;
}
if (state == IDLE && c >= '0' && c <= '6' && !ioFault) {
    demoCount = c - '0'; reason = "demo_count_changed"; changed = true; return;
}
if (c == 's' && LESSON_STAGE == 3 && state == IDLE && !ioFault) {
    startedAt = now; state = RUNNING; reason = "serial_start"; changed = true;
    advanceTimer(now); return;
}
if (hasRgb() && state == IDLE && !ioFault && (c == 'r' || c == 'g' || c == 'b' || c
== 'o')) {
    rgb(c == 'r', c == 'g', c == 'b');
    Serial.printf("event_type=rgb_test requested=%c visual_verified=false\n", c);
    return;
}
Serial.printf("event_type=command_rejected command=%c state=%s\n", c, stateName());
}
void drawSnapshot(uint32_t now) {
    if (!hasOled() || ioFault) return;
    if (!oledAck()) { ioFault = true; abortTimer(millis(), "display_ack_failed"); return;
}
    char text[24];
    oled.clearBuffer();
    oled.setFont(u8g2_font_6x12_tf);
    oled.drawStr(0, 12, stateName());
    snprintf(text, sizeof(text), "TIME %lu s", (unsigned
long)displaySeconds(remainingAt(now)));
    oled.drawStr(0, 28, text);
    if (state == RUNNING) oled.drawStr(0, 44, greenPhase ? "GREEN / COVER" : "RED /
WAIT");
```

下一頁接續同一支程式，不另開草稿。

week05_rgb_oled_timer

第 153 ~ 180 行。所有分頁合起來才是完整程式。

```
else {
    snprintf(text, sizeof(text), "RGB %s", rgbRequested);
    oled.drawStr(0, 44, hasRgb() && state == IDLE ? text : "NOT RUNNING");
}
snprintf(text, sizeof(text), "DEMO %d/6 NOT SCORE", demoCount);
oled.drawStr(0, 60, text);
oled.sendBuffer(); // Finite synchronous bus transfer, NOT zero-duration.
if (!oledAck()) { ioFault = true; abortTimer(millis(), "display_ack_failed"); }
}

void setup() {
    Serial.begin(115200); delay(1000);
    if (!profileReady()) { Serial.println("week=5 status=blocked
reason=profile_missing"); return; }
    ready = true;
    if (hasRgb()) {
        // Set the latch before output mode to reduce startup glitches after setup.
        rgb(false, false, false);
        pinMode(PIN_RGB_R, OUTPUT); pinMode(PIN_RGB_G, OUTPUT); pinMode(PIN_RGB_B, OUTPUT);
    }
    if (hasOled()) {
        if (!Wire.begin(PIN_SDA, PIN_SCL, 100000)) {
            ioFault = true; abortTimer(millis(), "i2c_begin_failed");
        } else {
            Wire.setTimeout(20);
            if (!oledAck()) { ioFault = true; abortTimer(millis(), "display_ack_failed"); }
            else {
                oled.setI2CAddress(OLED_ADDRESS_7BIT * 2);
                oled.setBusClock(100000); oled.begin(); Wire.setTimeout(20);
                if (!oledAck()) { ioFault = true; abortTimer(millis(), "display_ack_failed"); }
            }
        }
    }
}
```

下一頁接續同一支程式，不另開草稿。

week05_rgb_oled_timer

第 181 ~ 204 行。所有分頁合起來才是完整程式。

```
    }  
  }  
}  
if (LIGHT_INTERFERENCE_TEST) {  
  analogReadResolution(12); analogSetPinAttenuation(PIN_LIGHT, ADC_11db);  
}  
Serial.println("week=5 commands=r,g,b,o,0..6,s,x,f,c,z baud=115200");  
}  
void loop() {  
  if (!ready) return;  
  uint32_t now = millis();  
  advanceTimer(now); // Deadline is evaluated before ordinary serial commands.  
  if (Serial.available()) handleCommand((char)Serial.read(), now);  
  if (now - lastDraw >= DISPLAY_INTERVAL_MS) {  
    lastDraw = now; drawSnapshot(now);  
  }  
  now = millis(); advanceTimer(now); // Refresh after potentially slow I2C work.  
  if (changed || now - lastLog >= LOG_INTERVAL_MS) {  
    lastLog = now; changed = false;  
    Serial.printf("device_id=%s event_type=timer_status state=%s remaining_ms=%lu "  
                  "seconds=%lu phase=%s io_ok=%s reason=%s uptime_ms=%lu\n",  
                  DEVICE_ID, stateName(), (unsigned long)remainingAt(now),  
                  (unsigned long)displaySeconds(remainingAt(now)),  
                  state == RUNNING ? (greenPhase ? "GREEN" : "RED") : "OFF",
```

下一頁接續同一支程式，不另開草稿。

week05_rgb_oled_timer

第 205 ~ 214 行。所有分頁合起來才是完整程式。

```
        ioFault ? "false" : "true", reason, (unsigned long)now);
    Serial.printf("event_type=view_snapshot demo_count=%d rgb_requested=%s
visual_verified=false\n",
        demoCount, rgbRequested);
    if (LIGHT_INTERFERENCE_TEST) {
        const int raw = analogRead(PIN_LIGHT);
        Serial.printf("event_type=interference_sample light_raw=%d rgb_requested=%s
uptime_ms=%lu\n",
            raw, rgbRequested, (unsigned long)millis());
    }
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。

button_oled_timer

第 1 ~ 34 行。所有分頁合起來才是完整程式。

```
#include <Arduino.h>
#include <Wire.h>
#include <U8g2lib.h>

const bool PROFILE_CONFIRMED = false;
const int PIN_ADD = -1, PIN_START = -1;
const int PIN_SDA = -1, PIN_SCL = -1;
const int OLED_ADDRESS = -1;
const uint32_t STEP_SECONDS = 10;
const uint32_t MAX_SECONDS = 60;
const uint32_t DEBOUNCE_MS = 40;
#ifdef OLED_CONTROLLER
#define OLED_CONTROLLER 1306
#else
#define OLED_CONTROLLER 1315
#endif
if (OLED_CONTROLLER == 1306)
  U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(
    U8G2_R0, U8X8_PIN_NONE, PIN_SCL, PIN_SDA);
elif (OLED_CONTROLLER == 1315)
  U8G2_SSD1315_128X64_NONAME_F_HW_I2C oled(
    U8G2_R0, U8X8_PIN_NONE, PIN_SCL, PIN_SDA);
else
  #error Use the verified 1306 or 1315 configuration.
#endif

bool ready = false, armed = false, running = false, ioFault = false;
int lastAdd = HIGH, lastStart = HIGH;
uint32_t changedAt = 0, startedAt = 0, lastDraw = 0;
uint32_t durationSeconds = STEP_SECONDS;
uint32_t remainingMs = STEP_SECONDS * 1000;
const char* state = "IDLE";

int buttonEvent(uint32_t now) {
  const int add = digitalRead(PIN_ADD), start = digitalRead(PIN_START);
  if (add != lastAdd || start != lastStart) {
```

下一頁接續同一支程式，不另開草稿。

button_oled_timer

第 35 ~ 67 行。所有分頁合起來才是完整程式。

```
    lastAdd = add; lastStart = start; changedAt = now;
}
if (add == LOW && start == LOW) armed = false;
if (uint32_t(now - changedAt) < DEBOUNCE_MS) return 0;
if (add == HIGH && start == HIGH) { armed = true; return 0; }
if (!armed) return 0;
armed = false;
return add == LOW ? 1 : 2;
}

void report(const char* event) {
    Serial.printf("event=%s state=%s duration_s=%lu remaining_ms=%lu\n",
                  event, state, (unsigned long)durationSeconds,
                  (unsigned long)remainingMs);
}

void updateTime(uint32_t now) {
    if (!running) return;
    const uint32_t elapsed = now - startedAt;
    const uint32_t total = durationSeconds * 1000;
    remainingMs = elapsed >= total ? 0 : total - elapsed;
    if (remainingMs == 0) { running = false; state = "DONE"; report("expired"); }
}

bool hasAck() {
    Wire.beginTransaction(OLED_ADDRESS);
    return Wire.endTransmission() == 0;
}

void displayFault() {
    running = false; ioFault = true; state = "FAULT";
    report("display_fault");
}
```

下一頁接續同一支程式，不另開草稿。

button_oled_timer

第 68 ~ 101 行。所有分頁合起來才是完整程式。

```
void setup() {
  Serial.begin(115200);
  const int pins[] = {PIN_ADD, PIN_START, PIN_SDA, PIN_SCL};
  if (!PROFILE_CONFIRMED || OLED_ADDRESS < 8 || OLED_ADDRESS > 119 ||
      STEP_SECONDS < 1 || MAX_SECONDS > 3600 || STEP_SECONDS > MAX_SECONDS) {
    Serial.println("event=blocked reason=check_configuration"); return;
  }
  for (int i = 0; i < 4; ++i) {
    if (pins[i] < 0) return;
    for (int j = 0; j < i; ++j) if (pins[i] == pins[j]) return;
  }
  pinMode(PIN_ADD, INPUT_PULLUP); pinMode(PIN_START, INPUT_PULLUP);
  Wire.begin(PIN_SDA, PIN_SCL); Wire.setTimeout(20);
  if (!hasAck()) { displayFault(); return; }
  oled.setI2CAddress(OLED_ADDRESS * 2);
  oled.setBusClock(100000); oled.begin(); Wire.setTimeout(20);
  oled.setFont(u8g2_font_6x12_tf);
  changedAt = millis(); ready = true;
  report("ready");
}

void loop() {
  if (!ready || ioFault) return;
  const uint32_t now = millis();
  updateTime(now);
  const int button = buttonEvent(now);
  if (button == 1 && !running) {
    durationSeconds = durationSeconds + STEP_SECONDS > MAX_SECONDS
      ? STEP_SECONDS : durationSeconds + STEP_SECONDS;
    remainingMs = durationSeconds * 1000; state = "IDLE"; report("adjust");
  } else if (button == 1) report("ignored_while_running");
  if (button == 2) {
    if (running) { running = false; state = "STOPPED"; report("stop"); }
  }
}
```

下一頁接續同一支程式，不另開草稿。

button_oled_timer

第 102 ~ 117 行。所有分頁合起來才是完整程式。

```
    else {
        running = true; startedAt = now; state = "RUNNING";
        remainingMs = durationSeconds * 1000; report("start");
    }
}
if (uint32_t(now - lastDraw) < 200) return;
lastDraw = now;
if (!hasAck()) { displayFault(); return; }
char line[32];
oled.clearBuffer(); oled.drawStr(0, 14, state);
snprintf(line, sizeof(line), "TIME %lu s",
          (unsigned long)((remainingMs + 999) / 1000));
oled.drawStr(0, 32, line);
oled.drawStr(0, 50, "ADD / START-STOP"); oled.sendBuffer();
updateTime(millis());
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。